Group Project Phase 1

Group B05

Julia Areal Marques
jumar26@student.sdu.dk

Sara Salhani
sasal26@student.sdu.dk

Fatema Faurok
fafar26@student.sdu.dk

November 8, 2026

Contents

1 Introduction to Program Structure **1**

2 Relax and Toppling Algorithm **1**

2.1 Toppling 1

2.2 Relaxation 2

3 Doctest **4**

4 History Visualisation **5**

4.1 Command Processing and Simulation State 5

4.2 Recording the History 5

4.3 Plotting the History 6

5 Observation from a Simulation Run **6**

Appendices **9**

A GitHub **9**

B Declaration **9**

1 Introduction to Program Structure

This project simulates the Abelian sandpile model, where grains are added to a grid and redistributed when a cell becomes unstable. The program consists of three modules: `sandpile.py`, `sandpile_cli.py`, and `sandpile_support`.

The `sandpile.py` module handles the rules and calculations of the simulation. It contains five functions. `create_pile()` creates an empty grid, and `add_grain()` adds one grain to a selected cell. When a cell contains four or more grains, `topple()` distributes the grains to its neighbours. The `relax()` function continues toppling unstable cells until the grid is stable, while `is_stable()` checks whether all cells contain at most three grains.

For example, this part of `relax()` checks for an unstable cell and performs a toppling:

```
if pile[i][j] >= 4:
    number_of_lost_grains += topple(pile, i, j)
    number_of_topplings += 1
```

The `sandpile_cli.py` module handles user commands and controls the simulation through `main()`. When the user enters `run N`, the program adds one grain to a randomly selected cell in each cycle and then calls `relax()`. It also keeps track of grains added, grains lost, and topplings. The results from each cycle are stored in lists and can be displayed in a graph using Matplotlib.

The provided `sandpile_support` package contains `__init__.py` and `view.py`. The `view.py` file contains `SandpileView`, which displays the sandpile using different colours for different grain heights. It also contains `validate_pile()`, which checks the grid dimensions and cell values.

The modules work together through `sandpile_cli.py`, which calls the functions in `sandpile.py` and sends the updated grid and counters to `SandpileView` using `view.update()`. This keeps the calculations, user commands, and visualisation separate, making the program easier to understand and test.

2 Relax and Toppling Algorithm

We define a pile as a grid of $m \times n$ cells, with m rows and n columns, where the function `create_pile` creates an instance of a pile. We model it as a nested list of nonnegative integers with the type alias `Pile = list[list[int]]`, where `pile[i]` is row i, and `pile[i][j]` is the number of grains in row i and column j. This follows the mathematical representation of a matrix M, where $M_{i,j}$ corresponds to `pile[i][j]`, with indices starting at 0. In the following, we elaborate on the toppling and relaxation algorithms.

2.1 Toppling

The `topple` function executes a single step of toppling at cell (i, j) and returns the number of lost grains, assuming that the cell is unstable ($M_{i,j} \geq 4$). It removes four grains from the toppled cell, and each orthogonal neighbour receives one of them. When a boundary cell topples, the

grains sent to neighbours that do not exist are lost at the edge of the pile. We can think of the toppling of a cell as follows.

Let M be an $m \times n$ matrix and let (i, j) be the cell in row i and column j that we want to topple, and suppose $M_{i,j} \geq 4$. Then M':

$$
\begin{aligned}
M'_{i,j} &= M_{i,j} - 4 && \text{(the toppled cell)} \\
M'_{i-1,j} &= M_{i-1,j} + 1 && \text{if } i > 0 \quad \text{(north neighbour)} \\
M'_{i+1,j} &= M_{i+1,j} + 1 && \text{if } i < m - 1 \quad \text{(south neighbour)} \\
M'_{i,j-1} &= M_{i,j-1} + 1 && \text{if } j > 0 \quad \text{(west neighbour)} \\
M'_{i,j+1} &= M_{i,j+1} + 1 && \text{if } j < n - 1 \quad \text{(east neighbour)}
\end{aligned}
$$

Whether a neighbour exists depends on the position of the cell relative to the edges of the pile. The north and west neighbours exist only if `i > 0` and `j > 0`, respectively, since index 0 is the first row and the first column, which form the north and west edges. In the code, m and n correspond to `len(pile)` and `len(pile[0])`, so the south and east neighbours exist only if `i < len(pile) - 1` and `j < len(pile[0]) - 1`, respectively, i.e. when the cell is not on the south or east edge.

```
    # if north neighbour exists
    if i > 0:
        pile[i - 1][j] += 1
    else:
        lost += 1

    # if south neighbour exists
    if i < len(pile) - 1:
        pile[i + 1][j] += 1
    else:
        lost += 1

    # if west neighbour exists
    if j > 0:
        pile[i][j - 1] += 1
    else:
        lost += 1

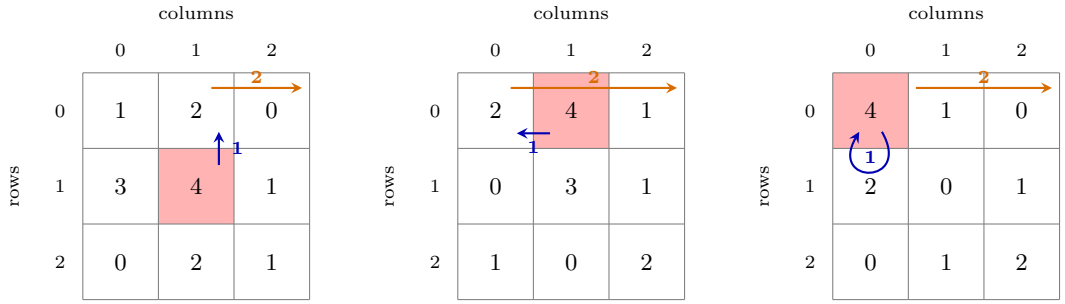
    # if east neighbour exists
    if j < len(pile[0]) - 1:
        pile[i][j + 1] += 1
    else:
        lost += 1
```

2.2 Relaxation

The `relax` function topples unstable cells until the pile is stable, and returns the number of topplings and the number of lost grains at the edges. It scans the pile in lexicographic order of (i, j), i.e. row by row from west to east, starting at $M_{0,0}$, and topples each unstable cell it meets. Since `topple` modifies the pile in place and returns the number of lost grains, `relax` only sums these and counts the topplings. However, a toppling adds grains to neighbours that

the scan may already have passed, so after each toppling the scan uses local backtracking to one of the predecessors: the north neighbour if it exists, and otherwise the west neighbour, since the successor is reached again as the scan continues in lexicographic order.

The figure below shows three scenarios. In **(a)**, if `i > 0`, we step north (`i -= 1`) to check the north predecessor. In **(b)**, if `i == 0` and `j > 0`, we step west (`j -= 1`), since this cell would not be checked again otherwise. In **(c)**, at $(0, 0)$ there is no predecessor, so we stay. If the cell is stable, the scan continues east (`j += 1`). At the end of a row (`j == len_column`) it moves to the next row (`i += 1`, `j = 0`).



(a) Topple, `i > 0`
1 backtrack north, `i -= 1`.
2 continue east along the row.

(b) Topple, `i == 0, j > 0`.
1 backtrack west, `j -= 1`.
2 continue east along the row.

(c) Topple at $(0, 0)$.
1 stay at $(0, 0)$.
2 continue east along the row.

The backtracking ensures that every predecessor cell that the toppled cell has redistributed grains to is rechecked. The only ones that come before it are north and west, which are rechecked as shown above, while the successor cells will be checked anyway. An excerpt of the `relax` function is provided below.

```
    while i < len_row:
        if pile[i][j] >= 4:
            number_of_lost_grains += topple(pile, i, j)
            number_of_topplings += 1

            # go back after toppling
            if i > 0:         # if north neighbour exists go north
                i -= 1
            elif j > 0:       # else if west
                j -= 1
            # else stay at (0, 0)

        else:
            j += 1            # go east
            if j == len_column:  # end of row
                j = 0
                i += 1
```

Hence, every cell before the scan position is stable, so when the last cell has been processed and the `while` loop terminates, the whole pile is stable. Since grains are lost at the edges, there are only finitely many topplings. The scan only moves forward, so the loop eventually terminates.

3 Doctest

We used doctests in `sandpile.py` to check whether the actual output of each function matches the expected output. In the subsequent we will elaborate on the doctests for `topple()` and `relax()`, since they are central to the sandpile simulation.

For the `topple()` doctest, we want to check whether the function distributes the grains correctly and counts the grains lost at the edges.

$$\begin{bmatrix} 4 & 1 \\ 2 & 2 \end{bmatrix} \xrightarrow{\text{topple (0,0)}} \begin{bmatrix} 0 & 2 \\ 3 & 2 \end{bmatrix}$$

The example above shows the calculation of the doctest: since $M_{0,0} \geq 4$, the cell topples and distributes one grain each to $(0, 1)$ and $(1, 0)$. The remaining two grains are lost at the north and west edges, since these neighbours do not exist, so `topple` returns `2`.

```
>>> pile = [[4, 1], [2, 2]]
>>> topple(pile, 0, 0)
2
>>> pile
[[0, 2], [3, 2]]
```

For the `relax()` doctest, we want to check whether the function topples unstable cells until the pile is stable, and returns the correct number of topplings and lost grains.

$$\begin{bmatrix} 1 & 4 & 3 \\ 2 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \xrightarrow{\text{topple (0,1)}} \begin{bmatrix} 2 & 0 & 4 \\ 2 & 3 & 1 \\ 1 & 1 & 1 \end{bmatrix} \xrightarrow{\text{topple (0,2)}} \begin{bmatrix} 2 & 1 & 0 \\ 2 & 3 & 2 \\ 1 & 1 & 1 \end{bmatrix}$$

As before, the example above shows the calculation of the doctest. The first toppling occurs at $(0, 1)$, since $M_{0,1} \geq 4$. It distributes one grain each to $(0, 0)$, $(0, 2)$ and $(1, 1)$, while one grain is lost at the north edge. Since the cell is on the first row, the scan steps west to $(0, 0)$, which is stable, and continues east. The grain received by $(0, 2)$ makes it unstable, so it topples as well and distributes one grain each to $(0, 1)$ and $(1, 2)$, while two grains are lost at the north and east edges. After this, all cells satisfy $M_{i,j} < 4$, so the pile is stable. Hence, the doctest expects two topplings and three lost grains.

```
    >>> pile = [[1, 4, 3], [2, 2, 1], [1, 1, 1]]
    >>> result = relax(pile)
    >>> result
    (2, 3)
    >>> pile
    [[2, 1, 0], [2, 3, 2], [1, 1, 1]]
```

When we ran the doctests in the terminal, all of them passed. These included the tests for `topple()`, `relax()` and the other functions in `sandpile.py`. The tests show that the functions give the expected results for the tested cases, but they do not cover every possible situation.

4 History Visualisation

To examine the sandpile's behavior over time, we record the topplings and grains lost during each simulation cycle. While `SandpileView` displays the current pile, the history visualisation shows the activity caused by individual grain additions.

4.1 Command Processing and Simulation State

The `main()` function initializes the pile, counters, and history lists, before entering a continuous command loop. User input is converted to lowercase and split into arguments, which are processed through an `if/elif` structure.

For example, the `run N` command requires one positive integer argument. The program checks argument count and uses `try/except` to handle invalid conversions, which prevents incorrect input from reaching the simulation. Since the data structures are initialized outside the command loop, their state persists between commands. The `reset` restores them to their initial state.

4.2 Recording the History

When the user enters `run N`, the program performs N cycles. In each cycle, a row and column are selected independently using `random.randrange()`, giving each cell an equal probability of receiving a grain.

After adding the grain, the pile is relaxed until all cells are stable. The returned toppling and grain-loss counts therefore represent the complete response to that grain addition. These values are appended to two lists as shown below.

```
row = random.randrange(rows)
column = random.randrange(columns)

sandpile.add_grain(pile, row, column)

cycle_topplings, cycle_lost = sandpile.relax(pile)

toppling_history.append(cycle_topplings)
lost_history.append(cycle_lost)
```

Unlike the two-dimensional list representing the pile, the histories are stored as two one-dimensional Python lists of integers, where each element represents the activity of one completed simulation cycle.

Let T_k and L_k denote the topplings and grains lost during cycle k, respectively. After N completed cycles, the histories are represented by

$$T = (T_1, T_2, \ldots, T_N), \qquad L = (L_1, L_2, \ldots, L_N)$$

Since Python uses zero-based indexing, T_k and L_k correspond to `toppling_history[k-1]` and `lost_history[k-1]`. Both lists receive one value per cycle, which preserves their correspondence. Storing individual rather than cumulative counts makes cycles with unusually high activity easier to identify.

4.3 Plotting the History

The `history` command uses Matplotlib to plot both lists against the cycle numbers, as shown below.

```
cycles = range(1, len(toppling_history) + 1)

fig = plt.figure("Sandpile History", figsize=(10, 7))
fig.clf()
ax1, ax2 = fig.subplots(2, 1, sharex=True)

ax1.plot(cycles, toppling_history, "k-")
ax2.plot(cycles, lost_history, "k--")
```

Separate vertical scales make grain loss easier to observe, while the shared horizontal axis allows the activity in the same cycle to be compared between the plots. Clearing the figure prevents overlapping plots, and non-blockingly display keeps the interface responsive. Since new data is appended to the history lists, multiple `run` commands extend the recorded history rather than replacing it.

5 Observation from a Simulation Run

We ran the simulation on a 20×20 grid for 1000 cycles, starting with an empty pile. Each cycle added one grain to a randomly selected cell, followed by relaxation. At the end of the simulation, 1000 grains had been added, 181 grains had been lost, and a total of 2891 topplings had occurred.

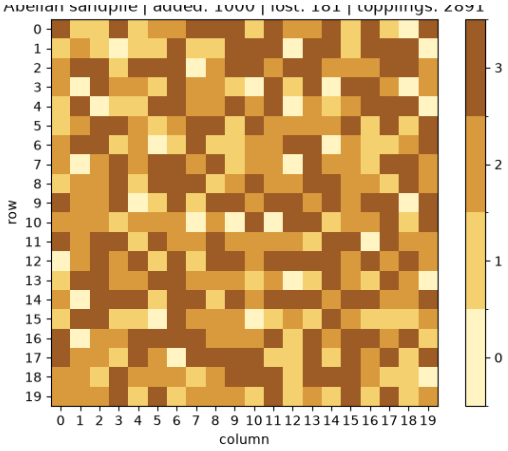


Figure 1: Final configuration of the 20×20 grid sandpile after 1000 simulation cycle. Colors represent cell heights from 0 to 3 grains.

Since the toppling only redistributes grains or removes them at the boundaries, the total number of grains is conserved except for the boundary losses. Thus, the 1000 added grains and 181 lost grains leave:

$$1000 - 181 = 819.$$

grains in the final pile.

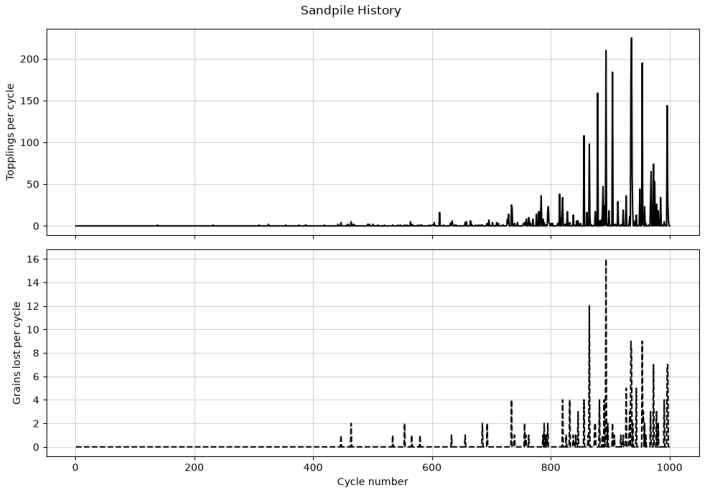


Figure 2: Topplings and grains lost per cycle during the 1000-cycle simulation on a 20×20 grid.

Figure 2 shows relatively little toppling activity during the first 300 cycles. Since the pile starts empty, many grain additions can occur without causing cells to become unstable at four grains. As the pile accumulates grains, topplings become more frequent.

After approximately 750 cycles, the activity becomes larger and more irregular, with the largest spike reaching about 225 topplings in one cycle. As described in the Toppling and Relaxation

section, a toppling redistributed grains to neighbouring cells, which potentially makes them unstable and triggers further topplings. Consequently, a single grain addition can produce a large avalanche, which explains he observed spikes in the history plot. Therefore, the activity depends on the current configuration of the pile.

Grain loss remains comparatively small, with the largest spike reaching approximately 16 grains lost. This is because only topplings at the grid boundaries can cause grain loss, whereas interior topplings redistribute grains within the pile. Consequently, many topplings do not necessarily imply substantial gran loss.

Appendices

A GitHub

The code can also be accessed through GitHub: `https://github.com/JulMarqz/sandpile-project`

B Declaration

Declaration of the Use of Generative Artificial Intelligence (GAI)
– valid version from 1 APRIL 2026

Use of GAI in assignments

As an implementation of the applicable rules and guidelines for the use of artificial intelligence at SDU, the Faculty of Science has designed a declaration form that you may attach as an appendix to written assignments and use to indicate whether you have used AI tools. It is voluntary to use the form, but if you choose not to use it, you must still ensure that your use of GAI tools is declared in accordance with SDU's applicable rules.

The declaration form may be attached to your submissions of all assignments submitted for assessment, except for written on-campus exams and other examinations where it is stated in the course description that GAI tools are not allowed.

If you *have* used GAI tools, you must state in the declaration form which tools you have used and how you have used them.

By declaring your use of GAI tools, you ensure that no issues arise in relation to the rules on cheating in exams, as SDU's rules stipulate that failure to disclose or indicate the use of GAI tools will be treated as plagiarism.

The assessment of the assignment is based solely on how you demonstrate fulfilment of the learning objectives in the course description. Correct and appropriate use of GAI tools may contribute to a higher degree of goal attainment, while inappropriate use may reduce the degree of goal attainment. You are encouraged to engage in dialogue with your lecturer or supervisor about how GAI tools can be used appropriately in your specific assignment and enhance your demonstration of the learning objectives.

Key points to note:

- Always remember to check the applicable rules and guidelines for the use of generative AI on mitsdu.dk.
- Be careful not to feed generative AI tools with personal data or other confidential information.
- If you have used generative AI tools to generate text, you must declare how you have used the tools. Remember to follow the rules for proper paraphrasing.
- If you have quoted directly from text generated by generative AI, you must indicate this as a quotation with a reference to the generative AI tool used – see guidance from the SDU Library.

How to declare the use of generative AI tools:

In your declaration of the use of generative AI tools, you must as a minimum:

1. state whether you have used generative AI or not,
2. if you have used generative AI, specify which technologies you have used,
3. describe how information has been generated, describe the inputs used, and explain how you used the output in your assignment.

You may use the following form when making your declaration:

Declaration for the use of generative AI tools

☐ **I/we have NOT used generative AI in the preparation of this assignment** *(tick the box).*

☒ **I/we have used generative AI in the preparation of this assignment** *(tick the box). List which generative AI tools have been used (remember to include the version).*

- ChatGpt (OpenAI, GPT-6)
- Claude (Anthropic, Opus 5.5)

I/we have used generative AI tools in the following way – *a list of possible uses is attached for inspiration.*

EXAMPLES OF USE
☐ Text generation and paraphrasing *In academic contexts, generative AI is not perceived as an authoritative source. It is therefore not recommended to use generative AI to generate finished text for academic assignments, unless the assignment requires it (e.g. assignments dealing with generative AI or prompt engineering).* *If you nevertheless insert text generated by generative AI into your assignment, it must be clearly declared:* *If you insert passages generated by generative AI directly into the assignment, they must be cited as a citation, and you must cite that generative AI on the list of references.* *Note: this also applies if you have used AI-generated notes that are included in the assignment.* *If you insert passages in which you have paraphrased generative AI output, or if you insert passages in which generative AI has paraphrased the original text, you must cite generative AI.* *Read more about citation of generative AI here (in Danish).*
☐ Structure *Generative AI has helped organise chapters and sections or other elements of the assignment.*
☐ Summarising large amounts of information *Generative AI has condensed large data sets.* *Document, using source references in the reference list, which original data have been summarised.*
☐ Analysis and/or interpretation of data, literature, theories, etc. *Generative AI has performed analyses and interpretations that are included in your assignment.* *Document, using source references in the reference list, which data have been analysed and/or interpreted.*
☐ Brainstorming and idea generation

Generative AI has helped generate new ideas and perspectives.
☐ Source and literature search *Generative AI was used to support your assignment, e.g. by identifying relevant sources, literature and materials.*
☒ Language proofreading or linguistic feedback *Generative AI has been used for proofreading (grammar, spelling and punctuation).* *Generative AI has contributed to the linguistic presentation of your assignment (e.g. correct use of subject-specific terminology, phrasing, language adjustments, language feedback, translation, etc.).*
☒ Generating images, maps, tables, audio, video, etc.
☒ Programming *Generative AI was used to work with programming.*
☐ Other use (please specify which): [Type here]

For each selected item, briefly explain how you have used the responses generated by the AI.
Describe, for example, how the information was generated, and explain how the output was used in your assignment.

We used AI to check grammar and improve the clarity of our writing. We also used AI to help write TikZ code in Overleaf to generate the Topple figures, which we had drawn ourselves beforehand.

Describe how you have critically assessed the responses provided by the AI.

We critically reviewed the responses from AI before using them to make sure they matched our own ideas and understanding. We also checked that the Topple figures matched our original drawings and what we had in mind.

